

ISMS 2.0 — COMPLETE TECHNICAL AUDIT & ARCHITECTURAL DOCUMENTATION

Target Feature: `src/app/features/forms` (TP/PIA One-Time Registration)

System: Integrated Scheme Management System (ISMS 2.0), Rajasthan

Audit Date: September 16, 2026

Target Directory: `d:\ISMS 2.0\ISMS2.0\src\app\features\forms`

Analysis Classification: Static Code Analysis, Architecture Inspection & Field-Level Audit (Zero-Modification Mode)

1. COMPLETE FORMS FOLDER STRUCTURE

The `src/app/features/forms` module houses the one-time registration workflow for Training Partners (TP) and Project Implementing Agencies (PIA). It contains 7 files across 3 subdirectories.

File Tree

d:/ISMS 2.0/ISMS2.0/src/app/features/forms/	
├ forms.routes.ts	(12 lines, 343 B)
├ components/	
│ └ tp-pia-registration/	
│ │ └ tp-pia-registration.component.html	(2,565 lines, 175,001 B)
│ │ └ tp-pia-registration.component.scss	(63 lines, 1,251 B)
│ │ └ tp-pia-registration.component.ts	(1,257 lines, 40,014 B)
├ models/	
│ └ tp-pia-registration.model.ts	(152 lines, 2,979 B)
└ services/	
│ └ form-validation.service.ts	(607 lines, 20,513 B)
│ └ tp-pia-registration.service.ts	(467 lines, 15,005 B)

File Details & Dependency Matrix

File Path	Type	Lines	Size	Primary Responsibility	Downstream
forms.routes.ts	TS (Routes)	12	343 B	Route definitions for forms module; defines FORMS_ROUTES with paths '' and 'tp-pia-registration'.	src/app/ (lazy loader)
models/tp-pia-registration.model.ts	TS (Models)	152	2,979 B	Central domain model definitions (TpPiaRegistrationData , sub-interfaces, enums).	Component Registration
services/form-validation.service.ts	TS (Service)	607	20,513 B	Synchronous field & tab validators, regex pattern library, error maps, submission trackers, toast messaging.	tp-pia-registrat
services/tp-pia-registration.service.ts	TS (Service)	467	15,005 B	Reactive state store for form data (signal<TpPiaRegistrationData>), auto-save debouncer, localStorage sync, sample mock data, application number generator, dropdown constant lists.	tp-pia-registrat
components/tp-pia-registration/tp-pia-registration.component.ts	TS (Component)	1,257	40,014 B	Master wizard controller. Manages 5 tabs, 5 searchable dropdowns, 4 modals, keyboard input filters, date formatters, file upload logic, step navigation, and final submission.	forms.ro
components/tp-pia-registration/tp-pia-registration.component.html	HTML (Template)	2,565	175,001 B	Monolithic template containing Rajasthan portal header, responsive stepper bar, 5 step cards, custom dropdowns, 4 dialog modals, mobile & desktop document tables, and review summary.	tp-pia-registrat
components/tp-pia-registration/tp-pia-registration.component.scss	SCSS (Styles)	63	1,251 B	Custom styling for scrollbar suppression (.no-scrollbar), smooth step transitions (.step-content-smooth), number input spinner suppression (.no-spinners), and dropdown scrollbars (.dropdown-menu-scroll).	tp-pia-registrat



2. MAIN COMPONENT — COMPLETE ANALYSIS

Component: TpPiaRegistrationComponent

Source Path: d:\ISMS 2.0\ISMS2.0\src\app\features\forms\components\tp-pia-registration\tp-pia-registration.component.ts

2.1 Imports Inventory & Rationale

Angular Core & Common Imports

1. **Component** (`@angular/core`): Decorator required to declare the class as an Angular UI component.
2. **inject** (`@angular/core`): Functional dependency injection function used to inject `TpPiaRegistrationService` and `FormValidationService` directly in property declarations without constructor boilerplate.
3. **signal** (`@angular/core`): Reactive state primitive used for local interactive states: `activeTab` , `showSuccessModal` , `fontScale` , accordion open/close flags, dropdown open flags, dropdown search query strings, drag-and-drop flags, and modal states.
4. **computed** (`@angular/core`): Derived reactive primitive used to calculate filtered dropdown option lists, tab-level error dictionaries, aggregate error collections, and tab completion/validity statuses.
5. **ChangeDetectionStrategy** (`@angular/core`): Configures the component to use `OnPush` change detection, minimizing template re-evaluations to signal changes and user interactions.
6. **HostListener** (`@angular/core`): Listens to window/document-level DOM events: `document:click` (to close searchable dropdowns when clicking outside) and `document:keydown.escape` (to dismiss dropdowns on Escape).
7. **OnInit** (`@angular/core`): Lifecycle interface; `ngOnInit()` verifies whether `data.basicInfo.applicationNo` exists and generates one if absent.
8. **CommonModule** (`@angular/common`): Provides core Angular structural directives including `ngClass` and pipes.
9. **FormsModule** (`@angular/forms`): Supplies `ngModel` and `ngModelChange` directives for two-way data binding across input, select, textarea, and checkbox controls.
10. **RouterModule** (`@angular/router`): Included in the component `imports` array for potential routing linkages (currently dormant in template).

Service & Constant Imports (`../../services/tp-pia-registration.service` , `../../services/form-validation.service`)

11. **TpPiaRegistrationService** : Primary state container service holding the reactive form data signal and localStorage synchronization routines.
12. **INDIAN_STATES** : Array of 29 Indian States used to populate searchable state dropdowns.
13. **RAJASTHAN_DISTRICTS** : Array of 33 Rajasthan Districts used for district search.
14. **BUSINESS_ACTIVITIES** : Array of 7 business activities for the organization profile select.
15. **ID_PROOF_TYPES** : Array of identity proof types for authorized person.
16. **COMMON_BANKS** : Array of 14 common commercial/scheduled banks for bank search.
17. **TRANSFER_MODES** : Array of 4 electronic transfer modes (`RTGS` , `NEFT` , `ECS` , `CBS`).
18. **ACCOUNT_TYPES** : Array of 2 bank account types (`Current Account` , `Savings Account`).
19. **FormValidationService** : Injected validation service providing synchronous validation rules, tab status tracking, and toast alerts.

Model Imports (`../../models/tp-pia-registration.model`)

20. **OfficerInCharge** : Model representing an administrative officer record.

21. `AwardItem` : Model representing an organization award/recognition.
22. `UploadedDocument` : Model representing compliance documents and their upload metadata.
23. `TpPiaRegistrationData` : Comprehensive aggregate model for the entire TP/PIA dossier.

2.2 Component Configuration & Metadata

Selector: `app-tp-pia-registration`

Standalone: `true`

ChangeDetection: `ChangeDetectionStrategy.OnPush`

Imports Array: `[CommonModule, FormsModule, RouterModule]`

Template URL: `./tp-pia-registration.component.html`

Style URL: `./tp-pia-registration.component.scss`

Lifecycle Hook Implementation: `OnInit` -> `ngOnInit()` initializes the application number if not present.

2.3 Reactive State Management & Component Properties

Property / Signal	Type	Initial Value	Nature	Purpose & Scope
<code>service</code>	<code>TpPiaRegistrationService</code>	Injected	Service	Central state and draft storage
<code>valService</code>	<code>FormValidationService</code>	Injected	Service	Validation engine
<code>Math</code>	<code>Math</code>	Native Math	Global	Exposes native <code>Math</code> to template
<code>activeTab</code>	<code>WritableSignal<number></code>	<code>1</code>	Signal	Currently active step (1 to 5)
<code>showSuccessModal</code>	<code>WritableSignal<boolean></code>	<code>false</code>	Signal	Toggles final submission success dialog
<code>fontScale</code>	<code>WritableSignal<'standard'></code>	<code>'large'</code>	<code>'xlarge'></code>	<code>'standard'</code>
<code>declarationAgreed</code>	<code>boolean</code>	<code>false</code>	Property	Checkbox state for Step 5 legal declaration
<code>isOrgProfileOpen</code>	<code>WritableSignal<boolean></code>	<code>true</code>	Signal	Accordion toggle for Step 1 Section 1.1
<code>isContactDetailsOpen</code>	<code>WritableSignal<boolean></code>	<code>false</code>	Signal	Accordion toggle for Step 1 Section 1.2
<code>isAddressRecordsOpen</code>	<code>WritableSignal<boolean></code>	<code>false</code>	Signal	Accordion toggle for Step 1 Section 1.2 (Address)
<code>isReviewStep10open</code>	<code>WritableSignal<boolean></code>	<code>true</code>	Signal	Step 5 review accordion for Organisation block

Property / Signal	Type	Initial Value	Nature	Purpose & Scope
isReviewOrgProfileOpen	WritableSignal<boolean>	true	Signal	Step 5 review sub-accordion for Profile
isReviewContactOpen	WritableSignal<boolean>	false	Signal	Step 5 review sub-accordion for Contact
isReviewAddressOpen	WritableSignal<boolean>	false	Signal	Step 5 review sub-accordion for Address
isReviewAuthPersonOpen	WritableSignal<boolean>	false	Signal	Step 5 review accordion for Authorized Person
isReviewBankOpen	WritableSignal<boolean>	false	Signal	Step 5 review accordion for Bank Details
isReviewDocsOpen	WritableSignal<boolean>	false	Signal	Step 5 review accordion for Documents
tabs	TabItem[]	5 items	Readonly Array	Tab metadata (id, label, shortLabel, icon)
currentStepInfo	{ title: string; subtitle: string }	Getter	Dynamic	Returns responsive step title and subtitle
data	TpPiaRegistrationData	Getter	Dynamic	Accessor for service.formData()
changeTimer	any	undefined	Timer	Debounce timer handle for auto-save dispatch (60ms)
tab1Errors	Signal<Record<string, string>>	Computed	Computed	Validation errors for Tab 1
tab2Errors	Signal<Record<string, string>>	Computed	Computed	Validation errors for Tab 2
tab3Errors	Signal<Record<string, string>>	Computed	Computed	Validation errors for Tab 3
tab4ProjectErrors	Signal<Record<string, string>>	Computed	Computed	Validation errors for Tab 4
errors	Signal<Record<string, string>>	Computed	Computed	Merged error dictionary for all fields
tabStatuses	Signal<Record<number, { isCompleted: boolean; isSubmittedInvalid: boolean }>>	Computed	Computed	Tab visual status map (valid, completed, invalid)
isDistrictDropdownOpen	WritableSignal<boolean>	false	Signal	District searchable dropdown open state
districtSearchQuery	WritableSignal<string>	''	Signal	District search text filter

Property / Signal	Type	Initial Value	Nature	Purpose & Scope
filteredDistricts	Signal<string[]>	Computed	Computed	Filtered list of districts matching query
isStateRegDropdownOpen	WritableSignal<boolean>	false	Signal	State Where Registered dropdown open state
stateRegSearchQuery	WritableSignal<string>	''	Signal	State Where Registered search text filter
filteredStatesReg	Signal<string[]>	Computed	Computed	Filtered list of states matching query
isStateAddrDropdownOpen	WritableSignal<boolean>	false	Signal	Address State/UT dropdown open state
stateAddrSearchQuery	WritableSignal<string>	''	Signal	Address State search text filter
filteredStatesAddr	Signal<string[]>	Computed	Computed	Filtered list of states matching query
isStateAuthDropdownOpen	WritableSignal<boolean>	false	Signal	Authorized Person State dropdown open state
stateAuthSearchQuery	WritableSignal<string>	''	Signal	Authorized Person State search text filter
filteredStatesAuth	Signal<string[]>	Computed	Computed	Filtered list of states matching query
isBankDropdownOpen	WritableSignal<boolean>	false	Signal	Bank Name searchable dropdown open state
bankSearchQuery	WritableSignal<string>	''	Signal	Bank Name search text filter
filteredBanks	Signal<string[]>	Computed	Computed	Filtered list of banks matching query
isBankDragging	WritableSignal<boolean>	false	Signal	Visual drag-over indicator for cheque dropzone
bankFileError	WritableSignal<string>	''	Signal	Error message for cheque upload
docErrorMessage	WritableSignal<string>	''	Signal	Error message for Step 4 document upload
previewingDoc	WritableSignal<UploadedDocument>	null>	null	Signal
showOfficerModal	WritableSignal<boolean>	false	Signal	Officer In-Charge CRUD modal open state
isOfficerEditing	WritableSignal<boolean>	false	Signal	Differentiates Add vs Edit mode for officer

Property / Signal	Type	Initial Value	Nature	Purpose & Scope
modalOfficerSubmitted	WritableSignal<boolean>	false	Signal	Flag tracking if officer save was attempted
isOfficerOpen	WritableSignal<boolean>	true	Signal	Dormant accordion flag for officer list
currentOfficer	OfficerInCharge	Blank Officer	Property	Model instance backing officer dialog
showAwardModal	WritableSignal<boolean>	false	Signal	Award Record CRUD modal open state
isAwardEditing	WritableSignal<boolean>	false	Signal	Differentiates Add vs Edit mode for award
modalAwardSubmitted	WritableSignal<boolean>	false	Signal	Flag tracking if award save was attempted
isAwardOpen	WritableSignal<boolean>	true	Signal	Dormant accordion flag for award list
currentAward	AwardItem	Blank Award	Property	Model instance backing award dialog
todayIso	string	Getter	Dynamic	Today's date (YYYY-MM-DD)
maxDobIso	string	Getter	Dynamic	Max DOB (18 years ago, YYYY-MM-DD)
minDobIso	string	Getter	Dynamic	Min DOB (100 years ago, YYYY-MM-DD)

2.4 Complete Method Inventory (64 Methods)

Method Name	Parameters	Return	Exact Purpose
ngOnInit	None	void	Initializes application number if not present
currentStepInfo (get)	None	{title, subtitle}	Returns responsive step header title & subtitle based on application state

Method Name	Parameters	Return	Exact Purpose
<code>data</code> (get)	None	<code>TpPiaRegistrationData</code>	Accessor for current form d object
<code>onDataChange</code>	None	<code>void</code>	Debounces updates (60ms) triggers auto-save
<code>getFieldValue</code>	<code>fieldKey: string</code>	<code>any</code>	Traverses dot-delimited prc path on form data
<code>isFieldInvalid</code>	<code>fieldKey: string</code>	<code>boolean</code>	Evaluates whether field has validation error and should display it
<code>getFieldError</code>	<code>fieldKey: string</code>	<code>string</code>	Returns error message strin invalid
<code>isTabValid</code>	<code>tabId: number</code>	<code>boolean</code>	Evaluates tab validity
<code>isTabCompleted</code>	<code>tabId: number</code>	<code>boolean</code>	Checks if tab is completed
<code>isTabSubmittedInvalid</code>	<code>tabId: number</code>	<code>boolean</code>	Checks if tab was submitted errors
<code>closeAllDropdowns</code>	None	<code>void</code>	Closes all 5 searchable dropdowns
<code>toggleDistrictDropdown</code>	<code>event?: MouseEvent</code>	<code>void</code>	Toggles district dropdown ; focuses search input
<code>selectDistrict</code>	<code>dist: string</code>	<code>void</code>	Selects district and closes n
<code>clearDistrict</code>	<code>event?: MouseEvent</code>	<code>void</code>	Clears selected district
<code>toggleStateRegDropdown</code>	<code>event?: MouseEvent</code>	<code>void</code>	Toggles registered state dropdown
<code>selectStateReg</code>	<code>state: string</code>	<code>void</code>	Selects registered state
<code>clearStateReg</code>	<code>event?: MouseEvent</code>	<code>void</code>	Clears registered state
<code>toggleStateAddrDropdown</code>	<code>event?: MouseEvent</code>	<code>void</code>	Toggles address state drop

Method Name	Parameters	Return	Exact Purpose
<code>selectStateAddr</code>	<code>state: string</code>	<code>void</code>	Selects address state
<code>clearStateAddr</code>	<code>event?: MouseEvent</code>	<code>void</code>	Clears address state
<code>toggleStateAuthDropdown</code>	<code>event?: MouseEvent</code>	<code>void</code>	Toggles authorized state dropdown
<code>selectStateAuth</code>	<code>state: string</code>	<code>void</code>	Selects authorized state
<code>clearStateAuth</code>	<code>event?: MouseEvent</code>	<code>void</code>	Clears authorized state
<code>toggleBankDropdown</code>	<code>event?: MouseEvent</code>	<code>void</code>	Toggles bank dropdown
<code>selectBank</code>	<code>bank: string</code>	<code>void</code>	Selects bank name
<code>clearBank</code>	<code>event?: MouseEvent</code>	<code>void</code>	Clears bank name
<code>onDocumentClick</code>	<code>event: MouseEvent</code>	<code>void</code>	Outside-click handler closes dropdowns
<code>onEscapePress</code>	None	<code>void</code>	Escape key listener closing dropdowns
<code>toggleSameAddress</code>	None	<code>void</code>	Syncs postal address with registered address
<code>onRegisteredAddressChange</code>	None	<code>void</code>	Triggers postal sync if <code>sameAsRegistered</code> is active
<code>todayIso</code> (get)	None	<code>string</code>	Returns current date in YYYY-MM-DD
<code>maxDobIso</code> (get)	None	<code>string</code>	Returns date 18 years ago in YYYY-MM-DD
<code>minDobIso</code> (get)	None	<code>string</code>	Returns date 100 years ago in YYYY-MM-DD
<code>onlyNumbers</code>	<code>event: KeyboardEvent</code>	<code>boolean</code>	Prevents non-digit keystrokes

Method Name	Parameters	Return	Exact Purpose
onlyLetters	event: KeyboardEvent	boolean	Allows letters, spaces, and c
onlyLettersSymbols	event: KeyboardEvent	boolean	Allows letters, spaces, and d
onlyAlphanumericSymbols	event: KeyboardEvent	boolean	Allows letters, numbers, sp / - .
onlyAlphanumeric	event: KeyboardEvent	boolean	Allows letters and numbers
onlyDecimals	event: KeyboardEvent, currentVal: any	boolean	Restricts input to digits and decimal point
sanitizeNumbers	val: any, maxLen?: number	string	Strips all non-digit characte
sanitizeLetters	val?: string, maxLen?: number	string	Strips non-letter/space/dot characters
sanitizeAlphanumericUpper	val?: string, maxLen?: number	string	Converts to uppercase alphanumeric
sanitizeAlphanumericSymbols	val?: string, maxLen?: number	string	Strips non-alphanumeric/sy characters
sanitizeAlphanumericSymbolsUpper	val?: string, maxLen?: number	string	Converts to uppercase alphanumeric with symbols
sanitizeLettersSymbolsUpper	val?: string, maxLen?: number	string	Converts to uppercase lette symbols (no numbers)
sanitizeDecimals	val: any, maxLen?: number	string	Strips invalid decimal forma
formatDateDisplay	dateStr?: string	string	Converts ISO YYYY-MM-DD DD/MM/YYYY
getIsoDate	dateStr?: string	string	Converts DD/MM/YYYY to I YYYY-MM-DD
onDateInput	`event: Event, field: 'registration'	'dob'	void

Method Name	Parameters	Return	Exact Purpose
<code>onDatePick</code>	<code>`event: Event, field: 'registration'</code>	<code>'dob'</code>	<code>void</code>
<code>openPicker</code>	<code>picker: HTMLInputElement</code>	<code>void</code>	Triggers browser native date picker UI
<code>onDobChange</code>	None	<code>void</code>	Calculates age based on DOB string
<code>onTypeIdProofChange</code>	None	<code>void</code>	Syncs <code>idNo</code> when ID type dropdown changes
<code>onAadhaarChange</code>	None	<code>void</code>	Sanitizes Aadhaar to 12 digits
<code>onVoterIdChange</code>	None	<code>void</code>	Uppercases Voter ID and syncs with <code>idNo</code>
<code>onPassportChange</code>	None	<code>void</code>	Uppercases Passport and syncs with <code>idNo</code>
<code>onBhamashahChange</code>	None	<code>void</code>	Syncs Bhamashah to <code>idNo</code>
<code>onPanChange</code>	None	<code>void</code>	Uppercases PAN and syncs <code>idNo</code>
<code>onBankDragOver</code>	<code>e: DragEvent</code>	<code>void</code>	Sets <code>isBankDragging</code> to true
<code>onBankDragLeave</code>	<code>e: DragEvent</code>	<code>void</code>	Sets <code>isBankDragging</code> to false
<code>onBankFileDrop</code>	<code>e: DragEvent</code>	<code>void</code>	Handles dropped cheque file
<code>onBankFileSelected</code>	<code>e: Event</code>	<code>void</code>	Handles selected cheque file input
<code>handleBankFile</code>	<code>file: File</code>	<code>void</code>	Validates size (5MB) & extension, cheque metadata
<code>removeBankFile</code>	None	<code>void</code>	Clears cheque metadata
<code>documents</code> (get)	None	<code>UploadedDocument[]</code>	Returns list of statutory documents
<code>totalRequiredCount</code> (get)	None	<code>number</code>	Counts mandatory documents

Method Name	Parameters	Return	Exact Purpose
<code>uploadedRequiredCount</code> (get)	None	<code>number</code>	Counts uploaded mandatory documents
<code>isTab4Submitted</code> (get)	None	<code>boolean</code>	Checks if Tab 4 was submitted
<code>isTab4Invalid</code> (get)	None	<code>boolean</code>	Checks if Tab 4 has missing required docs
<code>onDocFileSelected</code>	<code>event: Event,</code> <code>docId: string</code>	<code>void</code>	Validates size/type and attaches document
<code>removeDocument</code>	<code>docId: string</code>	<code>void</code>	Detaches document and re-pending
<code>showOfficerModal</code> (get)	None	<code>boolean</code>	Returns officer modal signal
<code>officers</code> (get)	None	<code>OfficerInCharge[]</code>	Returns officers array from data
<code>isOfficerTabInvalid</code> (get)	None	<code>boolean</code>	Evaluates officer tab validity
<code>officerErrors</code> (get)	None	<code>Record<string, string></code>	Validates <code>currentOfficer</code>
<code>isOfficerModalFieldInvalid</code>	<code>field: keyof OfficerInCharge</code>	<code>boolean</code>	Checks if officer dialog field invalid
<code>getOfficerModalFieldError</code>	<code>field: keyof OfficerInCharge</code>	<code>string</code>	Returns error string for officer field
<code>openAddOfficerModal</code>	None	<code>void</code>	Opens dialog in Add mode
<code>editOfficer</code>	<code>off: OfficerInCharge</code>	<code>void</code>	Opens dialog in Edit mode
<code>deleteOfficer</code>	<code>id: string</code>	<code>void</code>	Confirms and deletes officer record
<code>saveOfficer</code>	None	<code>void</code>	Validates and saves officer data array
<code>closeOfficerModal</code>	None	<code>void</code>	Closes officer dialog
<code>getEmptyOfficer</code>	None	<code>OfficerInCharge</code>	Returns blank officer data template
<code>awards</code> (get)	None	<code>AwardItem[]</code>	Returns awards array from data
<code>awardErrors</code> (get)	None	<code>Record<string, string></code>	Validates <code>currentAward</code>

Method Name	Parameters	Return	Exact Purpose
isAwardModalFieldInvalid	field: keyof AwardItem	boolean	Checks if award dialog field invalid
getAwardModalFieldError	field: keyof AwardItem	string	Returns error string for award field
openAddAwardModal	None	void	Opens dialog in Add mode
editAward	award: AwardItem	void	Opens dialog in Edit mode
deleteAward	id: string	void	Confirms and deletes award record
onAwardFileSelected	event: Event	void	Attaches file name to currentAward
saveAward	None	void	Validates and saves award to array
closeAwardModal	None	void	Closes award dialog
getEmptyAward	None	AwardItem	Returns blank award data template
scrollStepIntoView	tabId: number	void	Smoothly scrolls stepper button into viewport
switchTab	tabId: number	void	Switches active tab, auto-saves draft, scrolls top
nextTab	None	void	Validates current step, displays toast if invalid, advances stepper
prevTab	None	void	Navigates to previous step, saves draft
populateDemo	None	void	Populates sample government demo data
openPreviewModal	None	void	Wrapper navigating to Step (Review)
openReviewModal	None	void	Wrapper navigating to Step (Review)
submitFinalApplication	None	void	Final submission verification modal display
confirmSubmit	None	void	Wrapper delegating to submitFinalApplication

Method Name	Parameters	Return	Exact Purpose
<code>printAcknowledgement</code>	None	<code>void</code>	Triggers browser native print window
<code>setFontScale</code>	<code>`scale: 'standard'`</code>	<code>'large'</code>	<code>'xlarge'</code>

2.5 Code Pattern Categorization & Identification

Wrapper / Delegation Methods:

`openPreviewModal()` -> simply calls `switchTab(5)` .

`openReviewModal()` -> simply calls `switchTab(5)` (duplicate of `openPreviewModal`).

`confirmSubmit()` -> simply calls `submitFinalApplication()` .

Dormant / Unused Controller Methods in HTML:

`openAddOfficerModal()` , `editOfficer()` , `deleteOfficer()` , `saveOfficer()` , `closeOfficerModal()`

`openAddAwardModal()` , `editAward()` , `deleteAward()` , `saveAward()` , `closeAwardModal()`

`populateDemo()`

`setFontScale()`

(Note: The modals for Officer and Award are written into the HTML lines 2216–2509, but there are zero buttons or triggers in the 5 main tabs that ever call `openAddOfficerModal()` or `openAddAwardModal()` . They are remnants from a previous tab-based architecture).

Repeated Dropdown Logic:

5 near-identical implementations for District, State Reg, State Addr, State Auth, and Bank Name (toggle, select, clear, outside-click, escape, search filter). Totaling ~150 lines of duplicate logic.

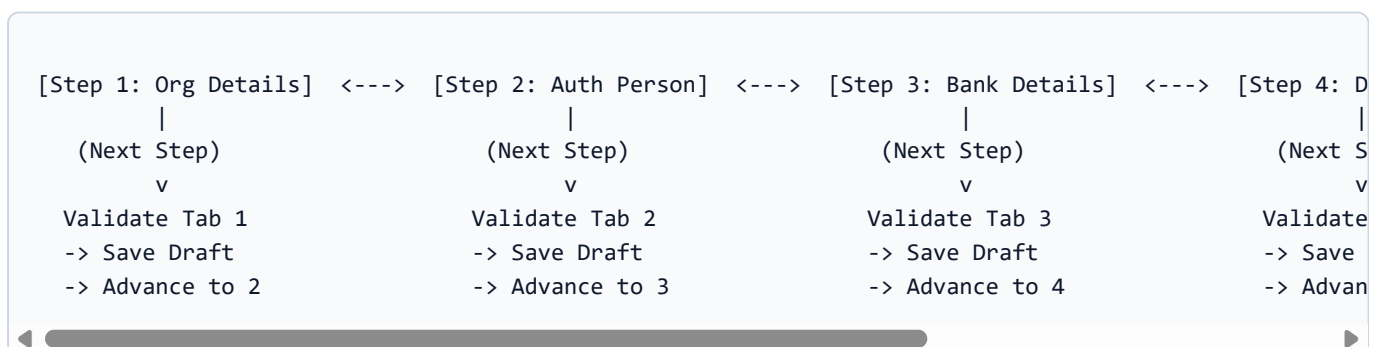
3. FORM WIZARD / STEP FLOW & NAVIGATION

The application implements a 5-step sequential wizard with non-blocking direct-tab jumping, validation gating on forward progression, auto-saving drafts on every tab switch, and automatic viewport repositioning.

3.1 Step Definitions & Metadata

Step ID	Label	Short Label	Icon	Header Title	Header Subtitle	Primary Data Domain
1	Organisation Details	Organisation		Step 1: Organisation / Company Basic Details	Primary profile, legal constitution, address records, and workflow authority	basicInfo , entityInfo , registeredAddress , postalAddress
2	Auth Person (Org)	Auth (Org)		Step 2: Authorized Person Details (Organisation Level)	Statutory corporate signatory, identity proofs, and legal credentials	authorizedOrg
3	Bank Details	Bank Details		Step 3: Bank Details	PFMS / DBT disbursal dedicated bank account, transfer mode, and verification records	bankDetails
4	Document Upload	Documents		Step 4: Document Upload	Mandatory statutory compliance documents, registration certificate, PAN, GST, and affidavits	documents
5	Review & Submit	Review		Step 5: Review & Final Submission	Review all application details, edit any section if needed, and submit the application	Readonly aggregate & declaration

3.2 Navigation Mechanics



1. Direct Tab Clicking (switchTab(tabId)):

Users can click any step button in the top stepper at any time.

If the current tab is valid, it marks it completed via `valService.markTabCompleted(current)`.

It performs a synchronous draft save via `service.saveDraftSync()` .

It updates `activeTab.set(tabId)` , clears active toasts, scrolls to top of page, and scrolls the active stepper button into view.

2. Next Step Button (`nextTab()`):

Evaluates on current `activeTab()` .

Marks the tab as submitted via `valService.markTabSubmitted(current)` .

If `current <= 4` and `!valService.isTabValid(current, this.data)` :

Triggers an error toast: `"Please fill all required fields correctly in \"{tabName}\" before proceeding."`

Aborts forward progression.

Automatically scrolls window to top to display error highlights.

If valid:

Marks current tab completed via `valService.markTabCompleted(current)` .

Clears toasts and runs `service.saveDraftSync()` .

Increments `activeTab` to `current + 1` .

Scrolls window to top and aligns stepper button in view.

3. Back Step Button (`prevTab()`):

Decrements `activeTab` by 1 if `activeTab() > 1` .

Disabled on Step 1.

Runs `service.saveDraftSync()` and smoothly scrolls to top.

4. Step Visual Indicator Logic (`tabStatuses` Computed):

Active Step: `bg-[#1a2656] text-white` (Navy Blue circle showing step number).

Completed Step: `bg-emerald-600 text-white` (Green circle showing checkmark ✓).

Submitted with Errors: `bg-rose-500 text-white` (Red circle showing exclamation !).

Untouched Step: `bg-white border-2 border-slate-300 text-slate-500` (Grey outlined circle showing step number).

4. EXHAUSTIVE FORM FIELD INVENTORY

The following matrix documents all **65 individual input controls, textareas, selects, dropdowns, checkboxes, and file dropzones** present across the 5 wizard steps and modal dialogs.

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
1	Step 1	1.1 Org Profile	Application No.	<code>data.basicInfo.applicationNo</code>	text	Opt	M

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
2	Step 1	1.1 Org Profile	Date of Registration	<code>data.basicInfo.dateOfRegistration</code>	text + date	Opt	M
3	Step 1	1.1 Org Profile	TP/PIA Full Name	<code>data.basicInfo.fullName</code>	text	Yes	3
4	Step 1	1.1 Org Profile	TP/PIA Short Name	<code>data.basicInfo.shortName</code>	text	Yes	2
5	Step 1	1.1 Org Profile	Entity Registration Number	<code>data.basicInfo.registrationNumber</code>	text	Opt	3
6	Step 1	1.1 Org Profile	Organisation PAN No.	<code>data.basicInfo.panNo</code>	text	Opt	M
7	Step 1	1.1 Org Profile	Type of Business / Activity	<code>data.entityInfo.businessActivity</code>	select	Opt	N,
8	Step 1	1.1 Org Profile	State Where Registered	<code>data.entityInfo.stateWhereRegistered</code>	custom dropdown	Opt	N,
9	Step 1	1.1 Org Profile	Annual Turnover (₹ in Lakhs)	<code>data.entityInfo.turnOver</code>	text (decimal)	Yes	M
10	Step 1	1.1 Org Profile	Official Website	<code>data.basicInfo.website</code>	url	Opt	M 1C
11	Step 1	1.2 Contact	Organisation Contact No.	<code>data.basicInfo.contactNo</code>	tel	Yes	M
12	Step 1	1.2 Contact	Company Official Email-ID	<code>data.basicInfo.emailId</code>	email	Yes	M
13	Step 1	1.2 Address	Premises / Building, Street & Area (Reg)	<code>data.registeredAddress.address</code>	textarea	Yes	M 25
14	Step 1	1.2 Address	State/UT (Registered)	<code>data.registeredAddress.state</code>	custom dropdown	Yes	N,
15	Step 1	1.2 Address	District (Registered)	<code>data.registeredAddress.district</code>	custom dropdown OR text	Yes	M

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
16	Step 1	1.2 Address	Pincode (Registered)	<code>data.registeredAddress.pincode</code>	text (num)	Yes	M
17	Step 1	1.2 Address	Same as Registered (Checkbox)	<code>data.sameAsRegistered</code>	checkbox	Opt	N,
18	Step 1	1.2 Address	Postal / Mailing Address	<code>data.postalAddress.address</code>	textarea	Cond.	M 25
19	Step 2	Personal Profile	Name (Authorized Person)	<code>data.authorizedOrg.name</code>	text	Yes	3
20	Step 2	Personal Profile	S/O, D/O, W/O (Guardian)	<code>data.authorizedOrg.guardianName</code>	text	Opt	M 10
21	Step 2	Personal Profile	Date of Birth	<code>data.authorizedOrg.dob</code>	text + date	Opt	M
22	Step 2	Personal Profile	Age (Years)	<code>data.authorizedOrg.age</code>	text (num)	Opt	M
23	Step 2	Personal Profile	Designation	<code>data.authorizedOrg.designation</code>	text	Opt	M
24	Step 2	Personal Profile	Residence Address	<code>data.authorizedOrg.residenceAddress</code>	textarea	Opt	M 25
25	Step 2	Contact & Proofs	Mobile No. (Authorized Person)	<code>data.authorizedOrg.contactNo</code>	tel	Yes	M
26	Step 2	Contact & Proofs	Email-Id	<code>data.authorizedOrg.emailId</code>	email	Opt	M
27	Step 2	Contact & Proofs	PAN Card No.	<code>data.authorizedOrg.pan</code>	text	Yes	M
28	Step 2	Contact & Proofs	Aadhaar No.	<code>data.authorizedOrg.aadhaarNo</code>	text (num)	Yes	M
29	Step 2	Contact & Proofs	State (Authorized Person)	<code>data.authorizedOrg.state</code>	custom dropdown	Opt	N,

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
30	Step 2	Contact & Proofs	Type ID Proof (Secondary)	<code>data.authorizedOrg.typeIdProof</code>	select	Opt	N,
31	Step 2	Contact & Proofs	Voter Id No. (Dynamic)	<code>data.authorizedOrg.voterIdNo</code>	text	Cond.	M
32	Step 2	Contact & Proofs	Passport No. (Dynamic)	<code>data.authorizedOrg.passportNo</code>	text	Cond.	M
33	Step 2	Contact & Proofs	Driving License No. (Dynamic)	<code>data.authorizedOrg.idNo</code>	text	Cond.	M
34	Step 2	Contact & Proofs	Bhamashah No. (Dynamic)	<code>data.authorizedOrg.bhamashahNo</code>	text	Cond.	M
35	Step 2	Contact & Proofs	Generic Secondary ID (Dynamic)	<code>data.authorizedOrg.idNo</code>	text	Cond.	M
36	Step 3	Bank Credentials	Name of the Bank	<code>data.bankDetails.bankName</code>	custom dropdown	Yes	N,
37	Step 3	Bank Credentials	Account No.	<code>data.bankDetails.accountNo</code>	text (num)	Yes	9
38	Step 3	Bank Credentials	IFSC Code	<code>data.bankDetails.ifscCode</code>	text	Yes	M
39	Step 3	Bank Credentials	Type of Account	<code>data.bankDetails.accountType</code>	select	Opt	N,
40	Step 3	Bank Credentials	Mode of electronic transfer	<code>data.bankDetails.electronicTransferMode</code>	select	Opt	N,
41	Step 3	Branch & Verification	Branch Name	<code>data.bankDetails.branchName</code>	text	Yes	M
42	Step 3	Branch & Verification	MICR Code	<code>data.bankDetails.micrCode</code>	text (num)	Opt	M
43	Step 3	Branch & Verification	Branch Address	<code>data.bankDetails.branchAddress</code>	textarea	Yes	M 25

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
44	Step 3	Branch & Verification	Cancelled Cheque Upload	<code>data.bankDetails.cancelledChequeFileName</code>	file dropzone	Opt	M 5M
45	Step 4	Compliance Documents	Doc 1: Registration Certificate	<code>documents['doc-reg-cert']</code>	file input	Yes	M 5M
46	Step 4	Compliance Documents	Doc 2: PAN Card	<code>documents['doc-pan']</code>	file input	Yes	M 5M
47	Step 4	Compliance Documents	Doc 3: GST Certificate	<code>documents['doc-gst']</code>	file input	Yes	M 5M
48	Step 4	Compliance Documents	Doc 4: Audited Balance Sheet	<code>documents['doc-turnover']</code>	file input	Opt	M 5M
49	Step 4	Compliance Documents	Doc 5: Cancelled Cheque / Bank Passbook	<code>documents['doc-bank']</code>	file input	Opt	M 5M
50	Step 4	Compliance Documents	Doc 6: Board Resolution / POA	<code>documents['doc-board-res']</code>	file input	Opt	M 5M
51	Step 4	Compliance Documents	Doc 7: NSDC Certificate	<code>documents['doc-nsdc']</code>	file input	Opt	M 5M
52	Step 4	Compliance Documents	Doc 8: Additional Supporting Doc	<code>documents['doc-other']</code>	file input	Opt	M 5M
53	Step 5	Legal Undertaking	Declaration Checkbox	<code>declarationAgreed</code>	checkbox	Yes	N,
54	Modal	Officer In-Charge	Officer Name	<code>currentOfficer.name</code>	text	Yes	M 1C
55	Modal	Officer In-Charge	Designation	<code>currentOfficer.designation</code>	text	Yes	M
56	Modal	Officer In-Charge	Mobile No.	<code>currentOfficer.mobileNo</code>	tel	Yes	M

#	Step / Location	Section / Container	Field Label	Exact Model Binding	Control Type	Req?	Le Li
57	Modal	Officer In-Charge	Email ID	<code>currentOfficer.emailId</code>	email	Yes	M
58	Modal	Officer In-Charge	PAN	<code>currentOfficer.pan</code>	text	Opt	M
59	Modal	Officer In-Charge	Aadhaar No.	<code>currentOfficer.aadhaarNo</code>	text (num)	Opt	M
60	Modal	Officer In-Charge	Bhamashah No.	<code>currentOfficer.bhamashahNo</code>	text	Opt	M
61	Modal	Officer In-Charge	Voter ID No.	<code>currentOfficer.voterIdNo</code>	text	Opt	M
62	Modal	Officer In-Charge	Passport No.	<code>currentOfficer.passportNo</code>	text	Opt	M
63	Modal	Award Record	Award / Recognition Name	<code>currentAward.awardName</code>	text	Yes	M 1C
64	Modal	Award Record	Awarding Agency / Body	<code>currentAward.awardingAgency</code>	text	Yes	M 1C
65	Modal	Award Record	Year of Award	<code>currentAward.year</code>	text (num)	Opt	M
66	Modal	Award Record	Level	<code>currentAward.level</code>	select	Opt	N,
67	Modal	Award Record	Description / Remarks	<code>currentAward.description</code>	text	Opt	M 2C
68	Modal	Award Record	Supporting Document	<code>currentAward.documentName</code>	file input	Opt	N,

5. TEMPLATE / HTML ARCHITECTURE & REPETITION ANALYSIS

The template (`tp-pia-registration.component.html`) is a **monolithic file of 2,565 lines and 175 KB**.

Line Breakdown by Major Structural Segment

Template Segment	Line Range	Line Count	Percent of File	Key Sub-components / Contents
1. Official Portal Header	1 – 33	33	1.3%	Ashoka Lion Emblem, State Titles, Bilingual Slogan, Saffron Stripe
2. Stepper Navigation Bar	34 – 85	52	2.0%	Responsive step circles, labels, connector lines, status badges
3. Form Title & Mandatory Legend	86 – 106	21	0.8%	Dynamic <code>currentStepInfo.title</code> , subtitle, mandatory indicator
4. Step 1: Organisation Details	107 – 760	654	25.5%	1.1 Profile (Application No, Date, Names, PAN, Turnover, Website), 1.2 Contact & Addresses (Phone, Email, Registered Address, Postal Address, 3 Searchable Dropdowns)
5. Step 2: Authorized Person	761 – 1157	397	15.5%	Personal Profile (Name, Guardian, DOB, Age, Designation, Address), Contact & Statutory Proofs (Mobile, Email, PAN, Aadhaar, State Dropdown, Dynamic ID field)
6. Step 3: Bank Details	1158 – 1473	316	12.3%	Bank Credentials (Searchable Bank dropdown, Account No, IFSC, Type, Mode), Branch & Verification (Branch Name, MICR, Address, Cheque Dropzone)
7. Step 4: Document Upload	1474 – 1737	264	10.3%	Upload counter banner, error alerts, Mobile Card View (<code>block md:hidden</code>), Desktop Table View (<code>hidden md:block</code>), 8 document rows with actions
8. Step 5: Review & Final Submission	1738 – 2107	370	14.4%	Collapsible review accordions for Step 1, 2, 3, 4, quick edit jump buttons, legal declaration checkbox, terms
9. Bottom Action Buttons	2108 – 2141	34	1.3%	Back button (<code>prevTab</code>), Next Step button (<code>nextTab</code>), Final Submit button (<code>submitFinalApplication</code>)
10. Dialog Modals	2142 – 2565	424	16.5%	Document Preview Modal (2146–2214), Officer In-Charge CRUD Modal (2217–2391), Award Record CRUD Modal (2394–2509), Submission Success Modal (2513–2563)

Redundant and Repeated Markup Patterns

1. Repeated Input Shell & Character Counter Pattern (Occurs 28 times):

```

<div class="flex items-center justify-between mb-1">
  <label class="block text-xs font-semibold text-slate-700">
    Field Label <span class="text-rose-500 font-bold">*</span>
  </label>
  <span class="text-[10px] text-slate-400 font-mono">{{ data.property.length || 0 }}/100</span>
</div>
<input ... [ngClass]="isFieldInvalid('...') ? 'border-rose-400 focus:border-rose-500 focus:ring-rose-400' : 'border-slate-200 focus:ring-slate-300'"
  class="w-full h-9 px-3 border rounded-md text-sm text-slate-800 placeholder-slate-400 hover:border-rose-400"
  @if (isFieldInvalid('...')) {
    <p class="text-xs text-rose-600 mt-1 font-medium">{{ getFieldError('...') }}</p>
  }

```

Every input field manually re-declares the flex label wrapper, character length calculation, ternary `ngClass` condition with hardcoded Tailwind strings, and `@if (isFieldInvalid())` error paragraph.

2. Repeated Searchable Dropdown Markup (Occurs 5 times, ~350 lines total):

Lines 270–343 (State Reg), lines 502–575 (State Addr), lines 578–662 (District), lines 989–1062 (State Auth), and lines 1178–1255 (Bank). Each declares identical button triggers, chevron SVG rotation, floating absolute panel (`top-full left-0 mt-1`), search input with magnifying glass SVG, clear button (`✕`), and `dropdown-menu-scroll` button list.

3. Duplicated Document Presentation Markup (Occurs twice in Step 4, ~240 lines):

The 8 documents are rendered twice: once in the mobile card list (`block md:hidden` , lines 1515–1616) and once in the desktop `<table>` view (`hidden md:block` , lines 1618–1734). Both iterate over `documents` and duplicate the file name, status badge, preview button, file input label, and remove button.

4. Duplicated Modal Wrappers (Occurs 4 times, ~400 lines):

Each modal declares an identical backdrop (`fixed inset-0 z-50 overflow-y-auto bg-slate-900/60 backdrop-blur-xs flex items-center justify-center p-3 sm:p-6`), an identical header with close button (`×`), and an identical footer button group.

6. SEARCHABLE DROPDOWNS DEEP DIVE & COMPARISON

The component implements 5 custom searchable dropdown components directly in the template and TypeScript.

Comparison Matrix

Dropdown Name	Bound Model Property	Source Array	Open Signal
State Where Registered	<code>data.entityInfo.stateWhereRegistered</code>	INDIAN_STATES (29)	<code>isStateRegDropdownOpen</code>
Address State/UT	<code>data.registeredAddress.state</code>	INDIAN_STATES (29)	<code>isStateAddrDropdownOpen</code>
Registered District	<code>data.registeredAddress.district</code>	RAJASTHAN_DISTRICTS (33)	<code>isDistrictDropdownOpen</code>
Auth Person State	<code>data.authorizedOrg.state</code>	INDIAN_STATES (29)	<code>isStateAuthDropdownOpen</code>
Bank Name	<code>data.bankDetails.bankName</code>	COMMON_BANKS (14)	<code>isBankDropdownOpen</code>



Identical Mechanism Across All 5 Dropdowns

- 1. Trigger:** A stylized `<button>` displaying the currently selected value or placeholder, with a clear icon (`×`) when populated, and a chevron SVG that rotates 180° when open.
- 2. Toggle:** Calls `toggleXDropdown($event)`, which calls `closeAllDropdowns()`, flips the signal, clears search query, and uses `setTimeout(..., 50)` to set DOM focus to the text input.
- 3. Popup Panel:** Positioned `absolute top-full left-0 mt-1 w-full min-w-[240px] z-50` with shadow and scrollable menu (`.dropdown-menu-scroll`).
- 4. Filtering:** An `@for` loop over the `computed()` signal which filters the source array case-insensitively (`items.filter(s => s.toLowerCase().includes(q))`).
- 5. Dismissal:** `@HostListener('document:click')` inspects `target.closest('.*-dropdown-container')`; `@HostListener('document:keydown.escape')` closes all 5 dropdowns.

7. ACCORDIONS & COLLAPSIBLE UI ELEMENTS

The component defines 10 reactive signals controlling collapsible UI panels.

Accordion Signal	Initial State	Controlling Trigger	Affected Template Block	Interactions & Anomalies
<code>isOrgProfileOpen</code>	<code>true</code>	Button at line 113	Step 1, Section 1.1 (Org Profile)	Independent accordion. Expanded by default.
<code>isContactDetailsOpen</code>	<code>false</code>	Button at line 394	Step 1, Section 1.2 (Contact & Address)	Coupled: The button simultaneously inverts BOTH <code>isContactDetailsOpen</code> and <code>isAddressRecordsOpen</code> .
<code>isAddressRecordsOpen</code>	<code>false</code>	Button at line 394	Step 1, Section 1.2 (Address Overflow)	Coupled: Toggled together with <code>isContactDetailsOpen</code> .
<code>isReviewStep10Open</code>	<code>true</code>	Header at line 1745	Step 5, Block 1 (Org Details Summary)	Parent accordion wrapping sub-accordions 1.1, 1.2, 1.3.
<code>isReviewOrgProfileOpen</code>	<code>true</code>	Header at line 1773	Step 5, Sub-block 1.1	Nested inside <code>isReviewStep10Open</code> .
<code>isReviewContactOpen</code>	<code>false</code>	Header at line 1817	Step 5, Sub-block 1.2	Nested inside <code>isReviewStep10Open</code> .
<code>isReviewAddressOpen</code>	<code>false</code>	Header at line 1842	Step 5, Sub-block 1.3	Nested inside <code>isReviewStep10Open</code> .
<code>isReviewAuthPersonOpen</code>	<code>false</code>	Header at line 1874	Step 5, Block 2 (Auth Person Summary)	Independent accordion in Step 5.
<code>isReviewBankOpen</code>	<code>false</code>	Header at line 1967	Step 5, Block 3 (Bank Details Summary)	Independent accordion in Step 5.
<code>isReviewDocsOpen</code>	<code>false</code>	Header at line 2044	Step 5, Block 4 (Documents Summary)	Independent accordion in Step 5.
<code>isOfficerOpen</code>	<code>true</code>	None (TS line 957)	None	Dormant: Declared in TypeScript, never bound in HTML.
<code>isAwardOpen</code>	<code>true</code>	None (TS line 1050)	None	Dormant: Declared in TypeScript, never bound in HTML.

8. MODALS ANALYSIS

Four distinct dialog modal structures are defined at the bottom of the HTML template (lines 2144–2565).

1. Document Preview Modal (`previewingDoc`)

Trigger: "Preview" button on any statutory document row in Step 4 (lines 1564, 1589, 1677, 1703).

Controlling Signal: `previewingDoc: WritableSignal<UploadedDocument | null>` .

Purpose: Displays document metadata (type, mandatory/optional, filename, upload date) and a mock document frame with a verified badge.

Dismissal: Top close button `×` or bottom "Close" button sets `previewingDoc.set(null)` .

2. Officer In-Charge CRUD Modal (`showOfficerModal`)

Trigger: `openAddOfficerModal()` or `editOfficer(off)` .

Controlling Signals: `showOfficerModal` , `isOfficerEditing` , `modalOfficerSubmitted` .

State Model: `currentOfficer: OfficerInCharge` .

Fields Inside Modal: Name (), *Designation* (), Mobile No (), *Email ID* (), PAN, Aadhaar No, Bhamashah No, Voter ID No, Passport No.

Save Routine (`saveOfficer`): Marks `modalOfficerSubmitted = true` , validates via `valService.validateOfficer(this.currentOfficer)` , appends to or updates `service.formData().officers` , closes modal.

CRITICAL ARCHITECTURAL FINDING: Orphaned Feature. While the modal HTML exists (lines 2217–2391) and the controller has full CRUD methods (`openAddOfficerModal` , `editOfficer` , `deleteOfficer` , `saveOfficer`), there are **zero buttons in the active 5 steps of the HTML that call** `openAddOfficerModal()` .

3. Award Record CRUD Modal (`showAwardModal`)

Trigger: `openAddAwardModal()` or `editAward(award)` .

Controlling Signals: `showAwardModal` , `isAwardEditing` , `modalAwardSubmitted` .

State Model: `currentAward: AwardItem` .

Fields Inside Modal: Award Name (), *Awarding Agency* (), Year of Award, Level (State/National/International), Description, Supporting Document file input.

Save Routine (`saveAward`): Marks `modalAwardSubmitted = true` , validates via `valService.validateAward(this.currentAward)` , appends to or updates `service.formData().awards` , closes modal.

CRITICAL ARCHITECTURAL FINDING: Orphaned Feature. Identical to the Officer modal, the Award modal HTML (lines 2394–2509) and controller methods exist, but there are **zero buttons in the active template that trigger** `openAddAwardModal()` .

4. Final Submission Success Modal (`showSuccessModal`)

Trigger: `submitFinalApplication()` upon successful validation and declaration agreement.
Controlling Signal: `showSuccessModal: WritableSignal<boolean>` .
Contents: Success checkmark icon, "Application Submitted!" heading, summary card with `applicationNo` , organisation full name, submission date, "Print Acknowledgement" button (`window.print()`), and "Close" button.

9. DOCUMENT & FILE UPLOAD SUBSYSTEM

The module handles file uploads across three distinct domains: Bank Cheque Verification (Step 3), Statutory Compliance Documents (Step 4), and Award Supporting Documents (Modal).

9.1 Upload Domain Specifications

Upload Domain	Bound Model Property	File Type Constraints	Max Size	UI Upload Mechanism	Handlers in Component
Cancelled Cheque (Step 3)	<code>bankDetails.cancelledChequeFileName</code> , <code>bankDetails.cancelledChequeFileSize</code>	<code>.pdf</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code>	5 MB	Drag & Drop Zone + Hidden File Input	<code>onBankDragOver()</code> , <code>onBankDragLeave()</code> , <code>onBankFileDrop()</code> , <code>onBankFileSelected()</code> , <code>removeBankFile()</code>
Statutory Documents (Step 4)	<code>data.documents[docId].fileName</code> , <code>fileSize</code> , <code>uploadDate</code> , <code>status</code>	<code>.pdf</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code>	5 MB	Individual "Browse" button per row in table & mobile cards	<code>onDocFileSelected(\$docId)</code> , <code>removeDocument(docId)</code>

Upload Domain	Bound Model Property	File Type Constraints	Max Size	UI Upload Mechanism	Handlers in Component
Award Document (Modal)	<code>currentAward.documentName</code>	<code>.pdf</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code>	5 MB	Standard file input button	<code>onAwardFileSelected(</code>

9.2 Statutory Compliance Document Inventory (Step 4)

Initial documents are seeded via `INITIAL_DOCUMENTS` in `tp-pia-registration.service.ts` :

Document ID	Document Type	Label / Description	Mandatory?	Default Status
<code>doc-reg-cert</code>	Registration Certificate	Organisation Registration Certificate	Mandatory (*)	pending
<code>doc-pan</code>	PAN Card	Organisation PAN Card	Mandatory (*)	pending
<code>doc-gst</code>	GST Certificate	GST Registration Certificate	Mandatory (*)	pending
<code>doc-turnover</code>	Audited Balance Sheet	Audited Balance Sheet / Turnover Certificate	Optional	pending
<code>doc-bank</code>	Bank Proof	Cancelled Cheque / Bank Passbook	Optional	pending
<code>doc-board-res</code>	Board Resolution	Board Resolution / Power of Attorney for Signatory	Optional	pending
<code>doc-nsdc</code>	NSDC Certificate	NSDC Partner Certificate (If Applicable)	Optional	pending
<code>doc-other</code>	Supporting Document	Additional Supporting Document	Optional	pending

9.3 Validation & Handling Pipeline

1. Size Validation: If `file.size > 5 1024 1024` (5 MB), an error is dispatched (`"filename exceeds the maximum 5 MB limit"`) and processing terminates.

2. Type Validation: Extracted extension is checked against `['pdf', 'jpg', 'jpeg', 'png']` . If invalid, error is set.

3. Metadata Calculation: Formats size string (`sizeFormatted = (file.size / (1024 * 1024)).toFixed(2) + ' MB'`) and date string (`dateFormatted = new Date().toLocaleDateString('en-GB')`).

4. State Storage: Note that **the actual binary File object is NOT stored in the frontend state or uploaded via HTTP multipart**. Only the metadata (`fileName` , `fileSize` , `uploadDate` , `status`) is written to `TpPiaRegistrationData` .

10. DATE HANDLING & DUAL-PICKER ARCHITECTURE

The application manages two date fields:

1. `data.basicInfo.dateOfRegistration` (Step 1)
2. `data.authorizedOrg.dob` (Step 2)

10.1 Dual-Picker Architecture

Each date input employs a dual-control strategy:

Visible Primary Input: A standard text input with a mask handler (`onDateInput`) allowing direct typing in `DD/MM/YYYY` format.

Hidden Native Date Input: An HTML5 `<input type="date">` element positioned absolutely with `opacity-0` `pointer-events-none` `w-0` `h-0` `-z-10` linked via template reference (`#regDatePicker` , `#dobPicker`).

Calendar Icon Button: Calls `openPicker(pickerRef)` , which attempts to invoke `picker.showPicker()` (modern HTML5 API) with fallback to `picker.focus(); picker.click()` .

10.2 Format Conversion Pipeline

Internal Storage Format: The form data stores dates as strings. Depending on user entry, it may be stored as `DD/MM/YYYY` (from text input) or `YYYY-MM-DD` (from sample data or native picker).

`formatDateDisplay(dateStr)` : Converts ISO `YYYY-MM-DD` strings to `DD/MM/YYYY` for template rendering.

`getIsoDate(dateStr)` : Converts `DD/MM/YYYY` strings to ISO `YYYY-MM-DD` for the hidden native picker's `[value]` attribute.

`onDateInput($event, field)` : Strips non-digits, slices to 8 digits, and dynamically injects slashes: `DD/MM/YYYY` .

10.3 Age Calculation Logic (`onDobChange`)

When the DOB input updates, `onDobChange()` parses the date (handling both `/` and `-` separators), compares it against the current system date, calculates the exact age in years, and updates `data.authorizedOrg.age` .

11. INPUT SANITIZATION & KEYBOARD EVENT RESTRICTIONS

The component employs a two-tier defense: keypress event suppression (preventing illegal characters from being typed) and input sanitization (cleaning pasted content or auto-fill).

Complete Sanitizer & Restriction Matrix

Function Name	Nature	Allowed Characters / Logic	Regex Pattern	Fields Utilizing
<code>onlyNumbers</code>	Keypress Event	Allows digits 0-9 and control keys (Backspace, Tab, Arrows, Enter, Delete)	<code>/^\d\$/</code>	Contact numbers, Aadhaar, Pincode, Account No, MICR, Age, Year
<code>onlyLetters</code>	Keypress Event	Allows letters (a-z, A-Z), spaces, and dots	<code>/^[a-zA-Z\s.]*\$/</code>	TP/PIA Full Name, Authorized Person Name, Guardian Name, Officer Name
<code>onlyLettersSymbols</code>	Keypress Event	Allows letters, spaces, and <code>/</code> , <code>&</code> , <code>-</code> , <code>.</code> (No digits)	<code>/^[a-zA-Z\s/&.-]*\$/</code>	TP/PIA Short Name
<code>onlyAlphanumericSymbols</code>	Keypress Event	Allows letters, digits, spaces, and <code>/</code> , <code>&</code> , <code>-</code> , <code>.</code>	<code>/^[a-zA-Z0-9\s/&.-]*\$/</code>	Registration No, Designation, Branch Name, Voter ID, Bhamashah
<code>onlyAlphanumeric</code>	Keypress Event	Allows strict letters and digits only	<code>/^[a-zA-Z0-9]*\$/</code>	PAN No, IFSC Code, Passport No
<code>onlyDecimals</code>	Keypress Event	Allows digits and at most ONE decimal point <code>.</code>	Digits + <code>'.'</code> check	Annual Turnover (<code>turnOver</code>)
<code>sanitizeNumbers</code>	Input Sanitizer	Strips all non-digit characters; enforces maxlength	<code>/\D/g</code>	Mobile No, Aadhaar No, Pincode, Bank Account No, MICR, Age, Year

Function Name	Nature	Allowed Characters / Logic	Regex Pattern	Fields Utilizing
<code>sanitizeLetters</code>	Input Sanitizer	Strips non-letters, spaces, and dots; enforces maxlength	<code>/[^\a-zA-Z\s.]/g</code>	Full Name, Guardian Name, Officer Name
<code>sanitizeAlphanumericUpper</code>	Input Sanitizer	Converts to uppercase; strips non-alphanumerics	<code>/[^A-Z0-9]/g</code>	PAN, IFSC, Passport
<code>sanitizeAlphanumericSymbols</code>	Input Sanitizer	Strips non-alphanumeric and symbols (& / - .)	<code>/[^\a-zA-Z0-9\s/&.-]/g</code>	Designation, Branch Name
<code>sanitizeAlphanumericSymbolsUpper</code>	Input Sanitizer	Uppercases; strips non-alphanumeric and symbols	<code>/[^A-Z0-9\s/&.-]/g</code>	Registration Number, Voter ID, Bhamashah
<code>sanitizeLettersSymbolsUpper</code>	Input Sanitizer	Uppercases; allows letters, spaces, symbols (no digits)	<code>/[^A-Z\s/&.-]/g</code>	Short Name
<code>sanitizeDecimals</code>	Input Sanitizer	Strips non-digits and multiple decimal points	Custom slice logic	Annual Turnover

12. VALIDATION ENGINE & RULES MATRIX

The validation subsystem is housed in `src/app/features/forms/services/form-validation.service.ts` and consumed by `TpPiaRegistrationComponent`.

12.1 Regular Expression Patterns Library (`VALIDATION_PATTERNS`)

```

export const VALIDATION_PATTERNS = {
  mobile: /^[6-9]\d{9}$/,
  email: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/,
  pan: /^[A-Z]{5}[0-9]{4}[A-Z]$/,
  gstin: /^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$/,
  pincode: /^\d{6}$/,
  aadhaar: /^\d{12}$/,
  ifsc: /^[A-Z]{4}0[A-Z0-9]{6}$/,
  micr: /^\d{9}$/,
  cin: /^[a-zA-Z0-9]{21}$/,
  bankAccount: /^\d{9,18}$/,
  url: /^(https?:\/\/)?([a-zA-Z0-9]([a-zA-Z0-9]*[a-zA-Z0-9])?\.\.)+[a-zA-Z]{2,}(\.[^\s]*)?$/i,
};

```

12.2 Validation Methods Matrix

Method Name	Target Data Entity	Evaluated Fields & Conditions	Validation Outcome / Error Message
validateTab1	basicInfo , entityInfo , registeredAddress , postalAddress	fullName : required, 3-100 chars, letters only
- shortName : required, 2-30 chars, letters/symbols
- registrationNumber : optional, 3-30 chars, alphanumeric
- website : optional, <=100 chars, valid URL
- dateOfRegistration : optional, DD/MM/YYYY, not future
- contactNo : required, valid 10-digit mobile
- emailId : required, valid email pattern
- panNo : optional, valid 10-char PAN
- registeredAddress.address : required
- registeredAddress.state : required
- registeredAddress.district : required
- registeredAddress.pincode : required, 6 digits
- turnOver : required, numeric >= 0, <= 10,000,000
- postalAddress.address : required if !sameAsRegistered	Returns TabValidationResult with populated errors dictionary.

Method Name	Target Data Entity	Evaluated Fields & Conditions	Validation Outcome / Error Message
<code>validateTab2</code>	<code>authorizedOrg</code>	- <code>name</code> : required, >=3 chars, letters only - <code>guardianName</code> : optional, letters only - <code>dob</code> : optional, DD/MM/YYYY, age >= 18 & <= 100 - <code>age</code> : optional, 18 <= age <= 100 - <code>contactNo</code> : required, valid 10-digit mobile - <code>pan</code> : required, valid 10-char PAN - <code>aadhaarNo</code> : required, exactly 12 digits - <code>emailId</code> : optional, valid email pattern	Returns <code>TabValidationResult</code> with populated <code>errors</code> dictionary.
<code>validateTab3</code>	<code>bankDetails</code>	- <code>bankName</code> : required, non-empty - <code>accountNo</code> : required, 9 to 18 digits - <code>ifscCode</code> : required, valid 11-char IFSC - <code>branchName</code> : required, non-empty - <code>branchAddress</code> : required, non-empty - <code>micrCode</code> : optional, exactly 9 digits	Returns <code>TabValidationResult</code> with populated <code>errors</code> dictionary.
<code>validateTab4</code>	<code>documents</code>	- Checks all documents where <code>required === true</code> - Verifies <code>status === 'uploaded'</code> and <code>fileName</code> present	Dispatches error if missing required docs count > 0.
<code>validateTab5</code>	Master Data	Delegates directly to <code>validateTab3(data)</code>	Matches Tab 3 validation.
<code>validateTab6</code>	<code>awards</code>	Iterates over <code>data.awards</code> and runs <code>validateAward()</code>	Dispatches error if any award has invalid details.
<code>validateTab7</code>	Master Data	Delegates directly to <code>validateTab4(data)</code>	Matches Tab 4 validation.
<code>validateOfficer</code>	<code>OfficerInCharge</code>	- <code>name</code> : required, letters only - <code>designation</code> : required - <code>mobileNo</code> : required, 10-digit mobile - <code>emailId</code> : required, valid email - <code>pan</code> : optional, valid PAN - <code>aadhaarNo</code> : optional, 12 digits	Used inside Officer Modal.
<code>validateAward</code>	<code>AwardItem</code>	- <code>awardName</code> : required - <code>awardingAgency</code> : required - <code>year</code> : optional, 4-digit year 1950–2100	Used inside Award Modal.
<code>validateDate</code>	Date string	Complete date parser and range validator	Validates format, valid days per month, leap years, min/max limits.

12.3 The Validation Cache Object Identity Bug

In `FormValidationService` (lines 568–592):

```
private validationCache = new Map<number, { dataRef: any; result: TabValidationResult }>();

getTabValidation(tabId: number, data: TpPiaRegistrationData): TabValidationResult {
  const cached = this.validationCache.get(tabId);
  if (cached && cached.dataRef === data) {
    return cached.result;
  }
  ...
}
```

CRITICAL FLAW: In `TpPiaRegistrationService`, `updateFormData` always creates a new object reference via object spread (`curr => ({ ...curr })`). Therefore, on every single keystroke, `dataRef === data` evaluates to `false`. The cache **never hits** during active typing, rendering the memoization logic completely ineffectual.

13. DATA MODEL AUDIT & DISCREPANCIES

The domain interfaces reside in `src/app/features/forms/models/tp-pia-registration.model.ts`.

13.1 Interface Structure

- `BasicOrgInfo` : Application metadata, entity name, registration number, contacts, PAN, CIN, GST.
- `EntityInfo` : Turnover, blacklisting status, NSDC partnership, nature of entity, applicant category, state, EOI details.
- `AddressInfo` : Address premises, state, district, pincode.
- `WorkflowInfo` : Internal department workflow routing (action, markTo, markToOfficer, remarks).
- `OfficerInCharge` : Administrative officer details.
- `AuthorizedPersonOrg` : Corporate statutory signatory details.
- `AuthorizedPersonProject` : Project-level authorized person details.
- `BankDetails` : Disbursal bank account, IFSC, branch, cheque metadata.
- `AwardItem` : Entity recognition details.
- `UploadedDocument` : Document metadata, upload date, status.
- `TpPiaRegistrationData` : Aggregate container aggregating all sub-interfaces.

13.2 Model Architectural Discrepancies & Stale Code

1. Mismatched Tab Number Comments:

Lines 122–146 of `tp-pia-registration.model.ts` list:

```
// Tab 1: basicInfo, entityInfo, registeredAddress, postalAddress, sameAsRegistered, workflowInfo
// Tab 2: officers
// Tab 3: authorizedOrg
// Tab 4: authorizedProject
// Tab 5: bankDetails
// Tab 6: awards
// Tab 7: documents
```

However, the active application has only **5 steps**. Tab 2 in the UI is actually `authorizedOrg`, Tab 3 is `bankDetails`, Tab 4 is `documents`, and Tab 5 is `Review & Submit`. The model comments reflect an obsolete 7-tab layout.

2. Orphaned `AuthorizedPersonProject` Interface:

The interface `AuthorizedPersonProject` (lines 76–84) and its corresponding property `authorizedProject` on `TpPiaRegistrationData` (line 137) are **completely absent from the HTML template**. No UI exists to view or edit project-level authorized persons.

3. Unused Properties on `BasicOrgInfo` & `EntityInfo`:

`cinNo` (`BasicOrgInfo`): Defined in model and sample data, but has no input field in the HTML template.

`gstNo` (`BasicOrgInfo`): Defined in model and sample data, but has no input field in Step 1 (GST Certificate is only uploaded as a document).

`blackListed`, `nsdcPartner`, `natureOfEntity`, `categoryOfApplicant`, `eoiReferenceNo`, `dateOfEoiPublished` (`EntityInfo`): Defined in model and populated by demo data, but omitted from Step 1 HTML form.

`workflowInfo` (`TpPiaRegistrationData`): Populated in demo data, completely omitted from UI.

14. REGISTRATION SERVICE AUDIT

The service `TpPiaRegistrationService` (`src/app/features/forms/services/tp-pia-registration.service.ts`) serves as the central data store.

14.1 State & Properties

`formData`: `signal<TpPiaRegistrationData>(this.getInitialState())` — Central reactive state for the entire form.

autoSaveStatus : `signal<'saved' | 'saving' | 'idle'>('idle')` — Tracks auto-save activity.
lastSavedTime : `signal<string>('')` — Formatted timestamp string of last save.
isSaving : `computed(() => this.autoSaveStatus() === 'saving')` — Readonly indicator for saving state.
lastSaveMessage : `signal<string>('')` — User-facing toast/status message.
autoSaveTimer : Internal timer handle for 400ms auto-save debounce.

14.2 Persistence Mechanism

Key: `'isms_tp_pia_registration_draft_v1'` .

Target Storage: Browser `localStorage` .

Auto-save Lifecycle:

`updateFormData()` triggers `triggerAutoSave()` .

Debounces for 400ms before invoking `saveDraftSync()` .

Window `beforeunload` listener invokes `saveDraftSync()` synchronously.

On service instantiation, constructor calls `restoreSavedDraft()` . If valid draft JSON exists, restores state into `formData` signal.

14.3 Static Constants Exported

Exports `INITIAL_DOCUMENTS` , `INDIAN_STATES` , `RAJASTHAN_DISTRICTS` , `NATURE_OF_ENTITIES` , `SCHEME_NAMES` , `BUSINESS_ACTIVITIES` , `APPLICANT_CATEGORIES` , `WORKFLOW_ACTIONS` , `MARK_TO_ROLES` , `ID_PROOF_TYPES` , `COMMON_BANKS` , `TRANSFER_MODES` , `ACCOUNT_TYPES` .

15. ROUTING ARCHITECTURE

Routing is declared in `src/app/features/forms/forms.routes.ts` :

```
import { Routes } from '@angular/router';
import { TpPiaRegistrationComponent } from '../components/tp-pia-registration/tp-pia-registration.component';

export const FORMS_ROUTES: Routes = [
  {
    path: '',
    component: TpPiaRegistrationComponent,
  },
  {
    path: 'tp-pia-registration',
    component: TpPiaRegistrationComponent,
  }
];
```

Integration: Lazy-loaded in `src/app/app.routes.ts` under `path: 'forms'` :

```
loadChildren: () => import('../features/forms/forms.routes').then(m => m.FORMS_ROUTES) .
```

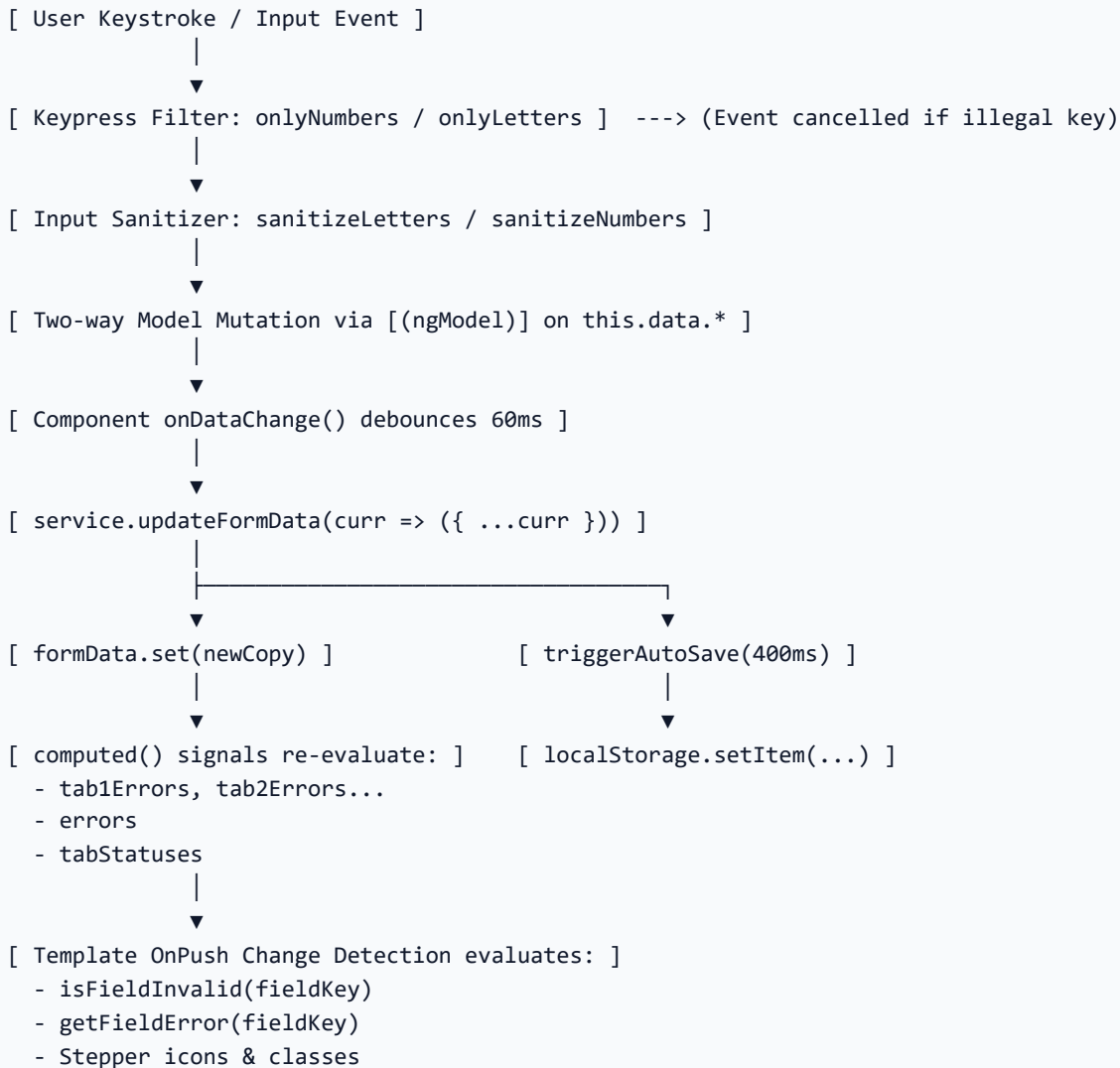
Accessible URLs:

`http://localhost:4200/forms`

`http://localhost:4200/forms/tp-pia-registration`

Route Guards / Resolvers: None. The route is completely open and public.

16. END-TO-END DATA FLOW ANALYSIS



Detailed Data Flow Stages

- 1. User Interaction & Keyboard Suppression:** Keystroke triggers `onlyNumbers()` or `onlyLetters()`. If regex check fails, `event.preventDefault()` halts entry before value reaches the DOM element.
- 2. Template Model Mutation:** Angular's `[(ngModel)]` directive directly writes the new primitive value into the nested property of `this.data` (e.g. `data.basicInfo.fullName`). Note that because `this.data` returns the mutable reference from the service's current signal, **the property is mutated in-place before `service.updateFormData()` is called.**
- 3. Component Change Debounce (`onDataChange`):** Clears and sets a 60ms timer. When the timer expires, it calls `service.updateFormData(curr => ({ ...curr })))`.
- 4. Service Signal Notification:** `service.updateFormData()` runs the callback, creating a shallow clone of the master object, and calls `this.formData.set(updated)`.
- 5. Auto-save Dispatch:** `service.triggerAutoSave()` debounces 400ms before stringifying the state and committing it to browser `localStorage` under key `'isms_tp_pia_registration_draft_v1'`.
- 6. Reactive Recomputation:** The modification of `formData` notifies downstream `computed()` signals: `tab1Errors`, `tab2Errors`, `tab3Errors`, `tab4ProjectErrors`, `errors` (aggregates the tab errors), `tabStatuses` (evaluates whether steps are completed or invalid).

7. Template OnPush Re-render: Angular re-evaluates bound template expressions (`isFieldInvalid()` , `getFieldError()` , `tabStatuses()`) and updates visual validation classes, error texts, and step badges.

17. PERFORMANCE & CHANGE DETECTION ANALYSIS

17.1 Confirmed Current Performance Bottlenecks & Architectural Bugs

#	Classification	Exact Issue	Technical Cause	Real-World Performance Impact
1	CONFIRMED	Validation Cache Ineffectiveness	<code>FormValidationService.getTabValidation</code> uses object identity comparison (<code>cached.dataRef === data</code>). Since <code>updateFormData</code> spreads <code>curr</code> into a new object on every change, the reference is always new.	Zero cache hits during typing. Validation algorithms run repeatedly and redundantly across change detection ticks.
2	CONFIRMED	Redundant Intermediate Computed Signals	Component defines <code>tab1Errors</code> , <code>tab2Errors</code> , <code>tab3Errors</code> , <code>tab4ProjectErrors</code> , and then merges them into <code>errors</code> .	Each of the 4 computed signals re-evaluates independently on every form data update, creating 5 computed signal recalculations per keystroke instead of 1.
3	CONFIRMED	Date Object Instantiation on Every Render	<code>todayIso</code> , <code>maxDobIso</code> , and <code>minDobIso</code> are implemented as getters creating <code>new Date()</code> instances every time they are evaluated in template attribute bindings.	Unnecessary garbage collection churn during typing and scrolling in Step 1 & 2.
4	CONFIRMED	Inline Regular Expression Re-compilation	Input restriction functions (<code>onlyNumbers</code> , <code>onlyLetters</code> , <code>onlyLettersSymbols</code> , <code>onlyAlphanumericSymbols</code> , etc.) construct new <code>RegExp</code> literal instances on every single keypress event.	Micro-overhead on every keyboard keystroke; should be static module-level constants.

#	Classification	Exact Issue	Technical Cause	Real-World Performance Impact
5	CONFIRMED	Monolithic Template DOM Size	A single 2,565-line HTML template with thousands of DOM nodes is maintained inside a single Angular component.	When change detection runs, Angular must traverse the entire component DOM tree, inspecting all 5 steps and 4 modals even though 4 steps and all modals are hidden.
6	CONFIRMED	Repeated String Splitting in Template Expressions	<code>isFieldInvalid(key)</code> and <code>getFieldError(key)</code> call <code>getFieldValue(key)</code> , which splits the key string on <code>'.'</code> and loops through properties on every change detection pass for dozens of fields.	Unnecessary string operations per field evaluated in the template.

17.2 Possible / Marginal Optimizations

Searchable Dropdown Filtering: The computed signals `filteredDistricts`, `filteredStatesReg`, etc. run `.filter()` on arrays with 14 to 33 items. Because the arrays are tiny, performance overhead is negligible, though extracting them to a shared component would dramatically clean up component code size.

Component Subtree Splitting: Splitting each of the 5 tabs into dedicated standalone child components (`app-tp-org-details`, `app-tp-auth-person`, `app-tp-bank-details`, `app-tp-documents`, `app-tp-review`) would restrict Angular's `OnPush` change detection specifically to the active tab's subtree.

17.3 Non-Issues / Patterns Functioning As Intended

`OnPush` Strategy: The component correctly uses `ChangeDetectionStrategy.OnPush`.

LocalStorage Performance: Auto-saving is properly debounced to 400ms, preventing main thread I/O thrashing during fast typing.

Signal Mutability: Despite the direct mutation of nested properties via `[(ngModel)]`, the 60ms debounce in `onDataChange()` ensures that Angular signals do receive new object references, triggering reactivity reliably.

18. CODE QUALITY & ARCHITECTURAL ASSESSMENT

18.1 Single Responsibility Principle (SRP) Violation

`TpPiaRegistrationComponent` is a classic "**God Component**" encompassing 1,257 lines of TypeScript and 2,565 lines of HTML. It simultaneously assumes responsibilities for:

1. Stepper wizard state and navigation logic
2. 5 distinct business domains (Organisation, Signatory, Bank, Documents, Review)
3. 5 separate searchable dropdown UI controllers (open, close, filter, clear, outside click, escape)
4. Character-level input filtering and regex sanitization
5. Dual-input date picker formatting and age calculation
6. File upload drag-and-drop and binary size validation
7. CRUD modal management for Officers and Awards
8. LocalStorage persistence coordination

18.2 Dormant & Orphaned Code

1. Officer In-Charge CRUD Subsystem: The component contains complete state management, validation, and HTML modal structure for Officers (`officers` , `currentOfficer` , `openAddOfficerModal` , `saveOfficer` , etc.), yet **no UI elements in the 5 steps allow users to view, add, or edit officers.**

2. Award Record CRUD Subsystem: Similarly, the Awards modal (`awards` , `currentAward` , `openAddAwardModal` , `saveAward` , etc.) is fully written into the component and template, but completely inaccessible in the main user journey.

3. Duplicate Wrapper Methods: `openPreviewModal()` , `openReviewModal()` , and `confirmSubmit()` are redundant passthroughs to `switchTab(5)` and `submitFinalApplication()` .

4. Dormant Variables: `isOfficerOpen` (line 957) and `isAwardOpen` (line 1050) are declared as signals but never bound in the template.

19. TEXTUAL DEPENDENCY & COUPLING MAP

```

src/app/app.routes.ts
| (loadChildren)
▼
src/app/features/forms/forms.routes.ts
|
▼
TpPiaRegistrationComponent
├─ TpPiaRegistrationService
│   ├── signal<TpPiaRegistrationData> (Master State)
│   ├── localStorage ('isms_tp_pia_registration_draft_v1')
│   └─ Static Constants (INDIAN_STATES, COMMON_BANKS, etc.)
├─ FormValidationService
│   ├── VALIDATION_PATTERNS (Regex library)
│   ├── Tab Validators (validateTab1, validateTab2, etc.)
│   ├── tabStatuses, submittedTabs, completedTabs
│   └─ toastMessage signal
├─ Data Models (tp-pia-registration.model.ts)
│   ├── TpPiaRegistrationData
│   ├── BasicOrgInfo, EntityInfo, AddressInfo
│   ├── AuthorizedPersonOrg, AuthorizedPersonProject
│   └─ BankDetails, AwardItem, UploadedDocument
└─ Template: tp-pia-registration.component.html (2,565 lines)
    ├── Header & Stepper
    ├── Step 1: Organisation Profile & Addresses (3 Dropdowns)
    ├── Step 2: Authorized Person Org (1 Dropdown, Dynamic IDs)
    ├── Step 3: Bank Details (1 Dropdown, Cheque Dropzone)
    ├── Step 4: Statutory Documents (Table & Mobile Cards)
    ├── Step 5: Review & Declaration
    └─ 4 Dialog Modals (Preview, Officer, Award, Success)

```

20. EXACT CURRENT USER FLOW & RUNTIME BEHAVIOR

Step-by-Step Runtime Walkthrough

1. Route Activation (/forms or /forms/tp-pia-registration):

forms.routes.ts initializes TpPiaRegistrationComponent .

In TpPiaRegistrationService constructor, restoreSavedDraft() checks localStorage for 'isms_tp_pia_registration_draft_v1' .

If a saved draft exists, it loads it into formData signal.

In ngOnInit() , if applicationNo is empty, it assigns a newly generated number (e.g. ISMS-TP-748291).

2. Step 1: Organisation Details:

Active step circle 1 is highlighted in Navy Blue (bg-[#1a2656]).

Section 1.1 (Profile) is open by default. User enters Full Name, Short Name, Registration Number, PAN, Turnover, and selects Business Activity and Registered State.

User clicks Section 1.2 header to expand Contact & Address.

User fills Contact No, Email, Registered Premises, District (searchable if Rajasthan), State, and Pincode.

Checking "Same as Registered" instantly copies Registered Address to Postal Address and disables Postal Address.

User clicks "Next Step".

`nextTab()` runs `valService.validateTab1()`. If any required field is invalid, an error toast appears at the top and the window smoothly scrolls to top.

If valid, Tab 1 is marked completed (✓ in green circle), draft is saved to `localStorage`, and wizard advances to Step 2.

3. Step 2: Authorized Person (Organisation Level):

Active step circle 2 is highlighted.

User enters Name (), Mobile (), PAN (), Aadhaar ().

User enters DOB. The input mask inserts slashes (DD/MM/YYYY). On blur, `onDobChange()` calculates the age in years.

User selects Secondary ID type (e.g. Voter ID). The dynamic input field appears below.

User clicks "Next Step". `valService.validateTab2()` validates mandatory fields. If valid, advances to Step 3.

4. Step 3: Bank Details:

Active step circle 3 is highlighted.

User clicks Bank Name button; searchable dropdown opens downwards. User searches "State Bank", selects SBI.

User inputs Account Number (), IFSC Code (), Branch Name (), Branch Address ().

User drags and drops a cancelled cheque PDF into the dropzone (or clicks to browse).

File handler validates that size <= 5 MB and extension is PDF/JPG/PNG. Card displays file name, formatted size, and "Ready" indicator.

User clicks "Next Step". Advances to Step 4.

5. Step 4: Statutory Document Upload:

Active step circle 4 is highlighted. Banner displays: "Uploaded: 0 / 3 Mandatory".

User browses and uploads Registration Certificate, PAN Card, and GST Certificate.

Counter updates to "Uploaded: 3 / 3 Mandatory".

User can click "Preview" on any document to inspect details inside the Document Preview modal.

User clicks "Next Step". Advances to Step 5.

6. Step 5: Review & Final Submission:

Active step circle 5 is highlighted.

User reviews collapsible accordion cards summarizing all entered details from Steps 1 to 4.

User can click "Edit Step X" to immediately jump back to that step.

User checks the mandatory "Declaration" checkbox.

"Submit Final Application" button becomes enabled.

User clicks "Submit Final Application".

`submitFinalApplication()` verifies that `declarationAgreed === true` and `getFirstInvalidTab() === null`.

Form data status is set to 'Submitted'.

`showSuccessModal` signal is set to `true`, displaying the submission success dialog.

User clicks "Print Acknowledgement" to invoke native browser print dialog, or clicks "Close".

21. POTENTIAL OPTIMIZATION & REFACTORING TARGETS

Note: As per strict task instructions, these are purely analytical recommendations. Zero code has been altered.

Group A: Safe / Low Risk Optimizations

1. Dormant Wrapper Method Elimination:

Current: `openPreviewModal()`, `openReviewModal()`, `confirmSubmit()` simply wrap other methods.

Target: Remove wrappers and bind directly to `switchTab(5)` and `submitFinalApplication()`.

Risk: Zero.

2. Static Date Getters to Readonly Class Properties:

Current: `todayIso`, `maxDobIso`, `minDobIso` are getters that execute `new Date()` on every template render cycle.

Target: Compute once in `readonly` class properties on component initialization.

Risk: Zero.

3. Static Regex Hoisting:

Current: Inline regexes like `/^\d$/`, `/^[a-zA-Z\s.]+$/` recompile on every keystroke.

Target: Hoist to module-level `const` declarations or import from a shared validation pattern file.

Risk: Zero.

4. Clean Stale Tab Comments in Data Model:

Current: Comments in `tp-pia-registration.model.ts` refer to 7 tabs.

Target: Update comments to match active 5-step architecture.

Risk: Zero.

Group B: Medium Risk Optimizations

5. Reusable Searchable Dropdown Component:

Current: 5 nearly identical searchable dropdowns are copy-pasted across the template and TS (~350 lines HTML, ~150 lines TS).

Target: Create a standalone `SearchableDropdownComponent` with `@Input() items`, `@Input() value`, and `@Output() valueChange`.

Benefit: Eliminates ~450 lines of duplicate markup and logic; unifies outside-click and keyboard navigation.

Risk: Low-Medium (requires testing two-way binding on all 5 dropdowns).

6. Fix Validation Cache Identity Check:

Current: `cached.dataRef === data` fails every time because of object spreading in `updateFormData`.

Target: Key cache by a change version counter or deep equality/stable hash.

Benefit: Restores intended performance caching behavior.

Risk: Low.

7. Consolidate Validation Computed Signals:

Current: 4 intermediate tab error computed signals merge into `errors`.

Target: Merge into a single computed signal evaluating the tabs directly.

Benefit: Reduces reactive graph complexity from 5 signals to 1.

Risk: Low.

Group C: Higher Risk / Architectural Refactoring

8. Split Monolithic Component into 5 Step Sub-components:

Current: 1,257-line TS file and 2,565-line HTML file in a single component.

Target: Decompose into:

- tp-pia-registration-shell.component (Stepper header, footer navigation, state orchestrator)
- step-org-details.component
- step-auth-person.component
- step-bank-details.component
- step-document-upload.component
- step-review-submit.component

Benefit: Adheres to Single Responsibility Principle, drastically improves readability, decouples change detection to active step subtrees, and simplifies unit testing.

Risk: Medium-High (requires careful input/output or shared service state alignment).

9. Eliminate or Integrate Dormant Officer & Award Subsystems:

Current: 300+ lines of HTML and 150+ lines of TS represent Officer and Award CRUD modals that cannot be triggered by users.

Target: Either wire up dedicated "Add Officer" and "Add Award" cards in Step 1/2, or cleanly prune them to dedicated sub-features.

Risk: Medium (requires product owner clarification on whether officers and awards are part of Phase 1 requirements).

22. EXECUTIVE SUMMARY & KEY METRICS

15 Critical Quantitative Metrics

#	Metric Description	Exact Value
:-:	:-:-	:-:-:
1	Total Files in src/app/features/forms	7 files
2	Total Components	1 component (TpPiaRegistrationComponent)
3	Total Services	2 services (TpPiaRegistrationService , FormValidationService)
4	Total Domain Model Files	1 file (tp-pia-registration.model.ts)
5	Total Defined Routes	2 routes ('' and 'tp-pia-registration')
6	Main Component TypeScript Line Count	1,257 lines (40,014 bytes)

#	Metric Description	Exact Value
7	Main Component HTML Template Line Count	2,565 lines (175,001 bytes)
8	Main Component SCSS Line Count	63 lines (1,251 bytes)
9	Number of Form Wizard Steps	5 steps
10	Approximate Total Field Count (including Modals & Dynamic fields)	68 distinct inputs / controls
11	Number of Custom Searchable Dropdowns	5 dropdowns (District, State Reg, State Addr, State Auth, Bank)
12	Number of Dialog Modals in Template	4 modals (Preview, Officer CRUD, Award CRUD, Success)
13	Number of Validation Engine Methods	10 methods in <code>FormValidationService</code>
14	Major Duplicated Logic Areas	4 areas (5x Dropdowns, 2x Document Views, 28x Input shells, 4x Modal shells)
15	Dormant / Orphaned Features Identified	2 complete CRUD subsystems (Officer In-Charge & Awards)

Top Architectural Observations

- 1. High Functional Completeness:** The forms feature contains rich government portal validation, thorough input masking (Aadhaar, PAN, IFSC, Mobile, Turnover, Dates), real-time auto-saving to localStorage, and responsive UI.
- 2. Extreme Component Density:** The main component violates Single Responsibility Principle by combining 5 complex domains, 5 custom searchable dropdowns, 4 modals, and all sanitizers in a single monolithic controller.
- 3. Orphaned Subsystems:** The Officer In-Charge and Award Record modals represent complete CRUD features that exist in both the TypeScript controller and HTML template, but lack entry-point trigger buttons in the active wizard steps.
- 4. Validation Cache Bug:** The memoization cache in `FormValidationService` fails 100% of cache checks during user typing due to object reference inequality caused by immutable state spreading.